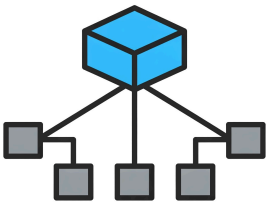
B 3.1.3 Static and Non-Static Python



B3.1.3 Distinguish between static and non-static variables and methods.

This section explains static vs. non-static (instance) variables and methods: their differences, usage, and scope. It covers when to use instance vs. class variables and how to apply these concepts in code.

Static and Non-Static Members in Python

This section of the eBook explores the distinction between static and non-static members (variables and methods) within a single class in Python, aligning with learning statement **B3.1.3** of the IB Computer Science syllabus. Grasping this concept is essential for writing clear and efficient Python programs using the principles of Object-Oriented Programming (OOP). Bear in mind that the creation of a class is covered in more detail in **B3.1.4** so you may want to revisit this section for more context.

Introduction to Static and Non-Static Members

In Python, a class acts as a template for creating objects. These objects encapsulate data (attributes, which are like variables) and behaviours (methods, which are like functions). Within a Python class, we can define attributes and methods that are either associated with the class itself (static or class-level) or with individual instances (objects) of the class (non-static or instance-level).

Non-Static Members (Instance Members)

Non-static attributes and methods are tied to the specific instances (objects) of a class. They represent the unique data and actions that each object possesses.

Non-Static Attributes (Instance Variables)

- **Usage and Scope:** Instance variables store data that is unique to each object. For instance, in a `Book` class, each book object would have its own title, author, and number of pages. These would be stored as instance variables. The scope of an instance variable is within the object it belongs to. It is typically accessed and modified through the object using the `self` keyword within instance methods.
- **Declaration:** Instance variables are usually defined within the constructor method (`__init__`) of the class using `self` to refer to the current object.

- **Code Example:**

```
class Book:
    def __init__(self, title, author, pages):
        self.title = title       # Instance variable
        self.author = author     # Instance variable
        self.pages = pages       # Instance variable
        self.is_borrowed = False # Another instance variable
    def borrow(self):
        if not self.is_borrowed:
            self.is_borrowed = True
            print(f"'{self.title}' has been borrowed.")
        else:
            print(f"'{self.title}' is already borrowed.")
    def display_details(self):
        print(f"Title: {self.title}, Author: {self.author}, Pages: {self.pages},
book1 = Book("The Hitchhiker's Guide to the Galaxy", "Douglas Adams", 224)
book2 = Book("Pride and Prejudice", "Jane Austen", 432)
book1.borrow()       # Output: 'The Hitchhiker's Guide to the Galaxy' has been b
book2.display_details() # Output: Title: Pride and Prejudice, Author: Jane Auste
print(book1.title)    # Accessing instance variable
print(book2.pages)    # Accessing instance variable
```

In this example, `title`, `author`, `pages`, and `is_borrowed` are instance variables. `book1` and `book2` are distinct `Book` objects, each holding its own set of these attributes. Operations on `book1`'s instance variables do not affect `book2`'s.

Non-Static Methods (Instance Methods)

- **Usage and Scope:** Instance methods operate on a specific object of the class. They have access to the instance variables of that object (using `self`) and can also access static (class-level) variables. They are used to perform actions that are relevant to the individual object's state or behaviour. Instance methods are called on an object using dot notation (e.g., `book1.borrow()`). The first parameter of an instance method is conventionally named `self`, which refers to the instance on which the method is called.
- **Declaration:** Instance methods are defined within the class and have `self` as their first parameter.
- **Code Example (continued from above):**

  The `borrow()` and `display_details()` methods in the `Book` class are instance methods. When called on `book1`, `self` refers to `book1`, and these methods can access and modify the instance variables of `book1`.

Static Members (Class Members)

Static attributes and methods belong to the class itself and are shared among all instances of the class. They exist independently of any particular object.

Static Attributes (Class Variables)

- **Usage and Scope:** Class variables store data that is common to all objects of the class. This can include default values, constants, or counters that track class-level information. Class variables are

defined within the class but outside of any instance methods (including `__init__`). They can be accessed using the class name (e.g., `Book.DEFAULT_GENRE`) or through an object of the class (e.g., `book1.DEFAULT_GENRE`). However, using the class name is the clearer and more conventional way to access class variables. Modifying a class variable through either the class or an object will affect all other objects of that class because they all share the same underlying variable. The scope of a class variable is within the class where it is defined.

- **Declaration:** Class variables are declared within the class definition but outside of any method.
- **Code Example:**

```
class Book:
    DEFAULT_GENRE = "Fiction"  # Class variable
    books_created = 0          # Class variable
    def __init__(self, title, author, pages):
        self.title = title
        self.author = author
        self.pages = pages
        self.is_borrowed = False
        Book.books_created += 1 # Increment class variable on object creation
    def get_genre(self):
        return Book.DEFAULT_GENRE
    @staticmethod
    def get_total_books_created(): # Static method (explained below)
        return Book.books_created
print(f"Default genre: {Book.DEFAULT_GENRE}") # Output: Default genre: Fiction
book1 = Book("The Lord of the Rings", "J.R.R. Tolkien", 1178)
book2 = Book("To Kill a Mockingbird", "Harper Lee", 281)
print(f"{book1.title}'s genre: {book1.get_genre()}")   # Output: 'The Lord of th
print(f"{book2.title}'s genre: {book2.get_genre()}")   # Output: 'To Kill a Mock
Book.DEFAULT_GENRE = "Literature"
print(f"{book1.title}'s genre after change: {book1.get_genre()}") # Output: 'The
print(f"Total books created: {Book.get_total_books_created()}")   # Output: Tota
print(f"Total books created (via object): {book1.get_total_books_created()}") #
```

Here, `DEFAULT_GENRE` and `books_created` are class variables. They are defined at the class level. When `DEFAULT_GENRE` is changed using `Book.DEFAULT_GENRE`, the change is reflected for all instances. `books_created` tracks the total number of `Book` objects.

Static Methods

- **Usage and Scope:** Static methods are associated with the class and not with any specific instance. They can be called directly on the class (e.g., `Book.get_total_books_created()`) and do not receive the instance as an implicit first argument (`self`). Static methods typically operate on class-level data or perform utility functions that are related to the class but don't require access to instance-specific data. They can access static variables but cannot directly access instance variables.
- **Declaration:** Static methods in Python can be defined using the `@staticmethod` decorator placed above the method definition [Not found in sources]. They do not have the `self` parameter.
- **Code Example (continued from above):**

  The `get_total_books_created()` method is a static method (indicated by `@staticmethod`). It belongs to the `Book` class and accesses the static variable `books_created`. It is called using the class name.

Try it yourself

Key Differences Summarised (Python)

Feature	Static Members (Class)	Non-Static Members (Instance)
Association	Class itself	Individual objects of the class
Memory	One copy shared by all objects	Each object has its own copy
Access	Class name or object name	Object name
Usage (Vars)	Common data for all objects	State specific to each object
Usage (Meths)	Operate on class-level data or utilities	Operate on the state of a specific object
Access to Inst. Members	Indirectly (through object parameters)	Directly (using `self`)

When to Use Instance Variables Instead of Class Variables

- **Use Instance Variables When:** You need to store data that is unique to each object. The value of the attribute will vary between different instances, representing their individual characteristics (e.g., a customer's address, a song's duration, a game character's health).
- **Do Not Use Class Variables When:** The data is not shared or common across all instances. If each object needs its own independent value for an attribute, it should be an instance variable defined using `self` within the `__init__` method (or other instance methods).

How to Apply These Concepts Effectively in Code (Python)

1. **Identify the Nature of the Data:** Decide if an attribute should hold the same value for all instances of the class or a unique value for each. Use a class variable (defined outside methods) for the former and an instance variable (defined within instance methods using `self`) for the latter.
2. **Consider the Method's Purpose:** If a method needs to work with the specific data of an object (its instance attributes), it should be an instance method and have `self` as its first parameter. If a method performs a general operation related to the class or only needs to interact with class-level attributes, it

can be a static method (using `@staticmethod` and without `self`). Utility functions that don't depend on the state of a particular object are often good candidates for static methods [Not found in sources].

3. **Access Class Members Clearly:** While you can access class variables and static methods through objects, it is more explicit and better practice to access them using the class name (e.g., `Book.DEFAULT_GENRE`, `Book.get_total_books_created()`). This clearly shows that they belong to the class.
4. **Initialise Instance Variables in `__init__`:** Ensure that instance variables are initialised within the `__init__` constructor. This sets up the initial state of each object when it is created.
5. **Manage Class Variable Modifications Carefully:** Because class variables are shared, be aware of where and how they are modified. Changes to a class variable will affect all existing and future instances of the class.

By properly distinguishing between and utilising static and non-static members in Python, you can create well-organised and efficient object-oriented programs that accurately model the relationships between classes and their instances.